

Cognitive Walkthrough

Evaluationskontext

<u>Software</u>	LLM Interaction Management System (LIMS)	
<u>Version</u>	1.0.0	
<u>Betriebssystem</u>	Windows 11 25H2	
<u>Nutzer</u>	Proband 3	
<u>Nutzerprofil</u>	<u>Erfahrung mit Programmierung</u>	<u>Erfahrung mit Python</u>
(Selbsteinschätzung)	9/10	9/10
<u>Testdatum</u>	27.07.2026	

T1	Endnutzerdokumentation aus dem GitHub-Repository beziehen
Ziel	Endnutzerdokumentation auf dem Zielsystem bereitgestellt, für weitere Hilfen geöffnet und als Anleitung für weitere Tasks bereit
Vorbedingung	Das GitHub-Repository ist bekannt
Schritt 1:	
Aktion	Der Nutzer bezieht die Endnutzerdokumentation aus dem Repository
Antwort	-
Nutzerverhalten	Der Nutzer öffnet das GitHub-Repository und folgt dem im Repository bereitgestellten Verweis auf die Endnutzerdokumentation. Er öffnet diese und hält sie für die Bearbeitung der nachfolgenden Aufgaben bereit.

T2	Software aus dem GitHub-Repository herunterladen und in der gewünschten Python-Umgebung installieren
Ziel	Die Software ist in der gewünschten Umgebung vorhanden und könnte in einem Projekt importiert werden
Vorbedingung	Die Nutzerdokumentation ist zur Unterstützung geöffnet und bereit, der Nutzer kann in der Dokumentation den Ablauf der Installation abrufen und nachvollziehen
Schritt 1: Befehl zur Installation ausführen	
Aktion	pip-Kommando aus der Endnutzerdokumentation herauslesen und in einer Eingabeaufforderung der Python-Umgebung ausführen
Antwort	-
Nutzerverhalten	Der Nutzer erstellt zunächst ein neues PyCharm-Projekt für die weitere Bearbeitung. Anschließend öffnet er die integrierte Konsole und führt den in der Endnutzerdokumentation angegebenen pip-Befehl zur Installation der Schnittstelle aus.
Schritt 2: Installation überprüfen	
Aktion	Installation beispielsweise durch eine Versionskontrolle in einem kleinen Codebeispiel überprüfen
Antwort	Python-Umgebung gibt Version des Tools aus
Nutzerverhalten	Der Nutzer überprüft die Installation direkt im neu erstellten Projekt, indem er die Schnittstelle importiert und über die Autovervollständigung in PyCharm kontrolliert, ob bereitgestellte Klassen und Funktionen verfügbar sind.

T3	Initialisierung der Software durch Objekt oder API
Ziel	Die Software ist in ihrem „Idle-Zustand“, mit den externen Handlern verbunden und der Nutzer hat entweder ein InteractionManager-Objekt oder eine Referenz zur High-Level API
Vorbedingung	T2 erfüllt
Schritt 1: Verbindungsstrukturen erstellen	
Aktion	Bei einer ersten Kommunikation mit den Endpunkten müssen die Datenstrukturen die Kommunikationsinformationen anbieten. Diese müssen vom Nutzer übergeben werden
Antwort	Keine Antwort des Systems
Nutzerverhalten	Der Nutzer legt die benötigten Verbindungsdaten anhand der in der Endnutzerdokumentation beschriebenen Struktur an.
Schritt 2: Verbindung initialisieren	
Aktion	Der Nutzer übergibt die Verbindungsinformationen an die Schnittstelle und startet den Verbindungsaufbau.
Antwort	Keine direkte Antwort des Systems
Nutzerverhalten	Der Nutzer erstellt auf Grundlage des Dokumentationsbeispiels ein InteractionManager-Objekt und übergibt die zuvor angelegten Verbindungsdaten an die Methode connect().
Schritt 3: Verbindung überprüfen	
Aktion	Nutzer verwendet eine der Methoden, beispielsweise is_connected(), um die Verbindung zu überprüfen
Antwort	System gibt Rückmeldung über Verbindungsstatus
Nutzerverhalten	Bei der Suche nach connect() erhielt der Nutzer über die Autovervollständigung auch is_connected() und erschließt daraus deren Funktion. Er verwendet die Methode eigenständig.

T4	Erneute Initialisierung basierend auf vorherigen Einstellungen
Ziel	Der Nutzer initialisiert die Software bei einer erneuten Verwendung aus den gespeicherten Verbindungsdaten, also ohne explizite Auswahl von Schnittstellen oder Angabe von Verbindungsinformationen
Vorbedingung	T2 erfüllt
Schritt 1: Verbindung initialisieren	
Aktion	Nutzer startet ohne Informationen einen Verbindungsaufbau
Antwort	Keine direkte Antwort des Systems
Nutzerverhalten	Der Nutzer orientiert sich an der Endnutzerdokumentation und initialisiert die Schnittstelle ohne erneute Übergabe der Verbindungsinformationen.
Schritt 2: Verbindung überprüfen	
Aktion	Nutzer verwendet eine der Methoden, beispielsweise is_connected(), um die Verbindung zu überprüfen
Antwort	System gibt Rückmeldung über Verbindungsstatus
Nutzerverhalten	Der Nutzer verwendet den bereits vorhandenen Aufruf von is_connected() zur Überprüfung des Verbindungsstatus.

T5	Prompt senden und Antwort erhalten
Ziel	Der Nutzer versteht die Interaktion zum Senden eines oder mehrerer Prompts zum LLM und kann die Antworten erhalten. Persistente Daten dieses Tasks sind ebenfalls für weitere Tasks erforderlich
Vorbedingung	Software initialisiert und verbunden – „Idle-Zustand“
Schritt 1: Konversation starten	
Aktion	Nutzer verwendet die Methode <code>start_conversation()</code> zum Starten einer Konversation
Antwort	Keine direkte Antwort des Systems
Nutzerverhalten	Der Nutzer erkennt anhand der Endnutzerdokumentation, dass vor dem Senden eines Prompts eine Konversation initialisiert werden muss. Er ergänzt seinen bestehenden Code gezielt um den Aufruf von <code>start_conversation()</code> .
Schritt 2: Prompt senden	
Aktion	Nutzer verwendet die Methode <code>send_prompt()</code> , um ein Prompt zu senden
Antwort	System sendet Antwort des LLM
Nutzerverhalten	Der Nutzer ergänzt den bestehenden Code um <code>send_prompt()</code> und führt diesen zunächst mit einem Breakpoint aus. Anhand des zurückgegebenen Objekts erkennt er unmittelbar die Struktur des Rückgabeobjekts. Anschließend gibt er beide Werte getrennt in der Konsole aus.

T6	Setzen eines Systemprompts, erneutes Senden eines Prompts
Ziel	Der Nutzer weiß, wie Einstellungen vorgenommen und ausgewählt werden, ein Systemprompt ist hinterlegt und dessen Einfluss kann bei einer weiteren Nachricht überprüft werden
Vorbedingung	Software initialisiert und verbunden – „Idle-Zustand“
Schritt 1: Hinterlegen eines Systemprompts	
Aktion	Hinzufügen von Daten zur Einstellung „system_prompt“
Antwort	Keine direkte Antwort des Systems
Nutzerverhalten	Der Nutzer sucht in der Endnutzerdokumentation nach der Einstellung <code>system_prompt</code> . Aus der Dokumentation erkennt er, dass <code>write_setting()</code> einen settings-Schlüssel sowie einen Wert erwartet. Er verwendet den Enum-Wert für die allgemeine Einstellungskonfiguration und setzt dort <code>system_prompt</code> auf den gewünschten Systemprompt.
Schritt 2: Prompt senden	
Aktion	Nutzer verwendet die Methode <code>send_prompt()</code> , um ein Prompt mit hinterlegtem Systemprompt zu senden
Antwort	System sendet Antwort des LLM
Nutzerverhalten	Der Nutzer verwendet seinen bestehenden Code zum Senden eines Prompts erneut. Anhand der zurückgegebenen Antwort überprüft er, ob der hinterlegte Systemprompt berücksichtigt wurde.

T7	Hinzufügen von Kontextdaten, erneutes Senden eines Prompts
Ziel	Der Nutzer weiß, wie komplexe Kontextdaten an den Prompt angefügt werden können und kann die Kontext-Einstellungen unterscheiden
Vorbedingung	Software initialisiert und verbunden – „Idle-Zustand“
Schritt 1: Kontextdaten hinzufügen	
Aktion	Hinzufügen von Kontextdaten durch die Methode <code>set_context_data()</code>
Antwort	Keine direkte Antwort des Systems
Nutzerverhalten	Der Nutzer sieht in der geöffneten Dokumentation neben <code>system_prompt</code> die Einträge <code>context_data</code> und <code>use_context_data</code> . Er hinterlegt die Kontextdaten in <code>context_data</code> vergleichbar zum <code>system_prompt</code> und setzt <code>use_context_data</code> über <code>set_context_mode()</code> auf <code>PERSISTENT</code> .
Schritt 2: Prompt senden	
Aktion	Nutzer verwendet die Methode <code>send_prompt()</code> , um ein Prompt mit hinterlegten Kontextdaten zu senden
Antwort	System sendet Antwort des LLM
Nutzerverhalten	Der Nutzer verwendet seinen bestehenden Code zum Senden eines Prompts erneut und überprüft anhand der Antwort, ob die hinterlegten Kontextdaten berücksichtigt wurden.

T8	Ähnlichkeitssuche auf der Vektordatenbank
Ziel	Der Nutzer weiß, wie eine Ähnlichkeitssuche auf Basis der vorherigen Ergebnisse durchgeführt wird. Die Antwort der Schnittstelle erscheint im Kontext der vorherigen Tasks subjektiv logisch
Vorbedingung	Software initialisiert und verbunden – „Idle-Zustand“, Daten vorhanden
Schritt 1: Ähnlichkeitssuche ausführen	
Aktion	Nutzer verwendet die Methode <code>nearest_search_vector</code> um eine Ähnlichkeitssuche auszuführen
Antwort	<code>top_k</code> ähnliche Einträge zur Eingabe werden aus der Vektordatenbank ausgegeben
Nutzerverhalten	Der Nutzer verwendet die bestehende Konversation für die Ähnlichkeitssuche und ergänzt den Aufruf um <code>nearest_search_vector()</code> . Über die Methodensignatur erkennt er die Bedeutung von <code>top_k</code> und wählt eigenständig einen passenden Wert. Anschließend setzt er einen Breakpoint, prüft die zurückgegebenen Ergebnisse und gibt diese separat in der Konsole aus.